

Sites LabMiM / LEAL

Onboarding da plataforma estática multi-publicação

Objetivo

Uma aplicação compartilhada para publicações meteorológicas de diferentes instituições, estados e produtos WRF.

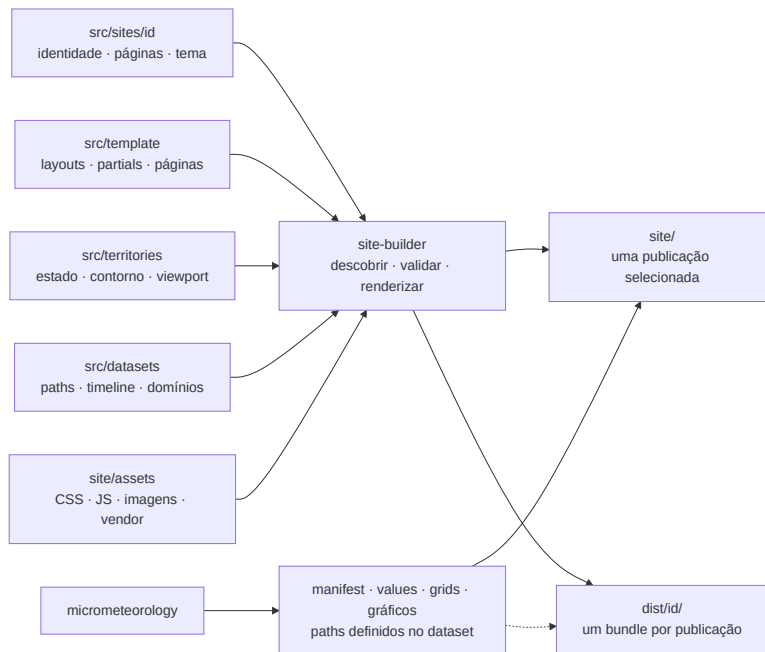
Hoje

- LabMiM / UFBA — Bahia
- LEAL / UFES — Espírito Santo
- Estrutura pronta para novas publicações

Camada	Escolha
Build	Node 24 + gerador próprio
Saída	HTML, CSS e JS puros
Mapa	Leaflet 1.9.4
Gráficos	Chart.js 3.9.1
Dados	JSON, GeoJSON e binários WRF

Node só existe no build. O servidor continua inteiramente estático.

Mapa do repositório



Fontes editáveis

`src/template/` · `src/sites/` · `src/territories/` ·
`src/datasets/`

Saídas geradas

`site/` — uma publicação · `dist/<id>/` — todas

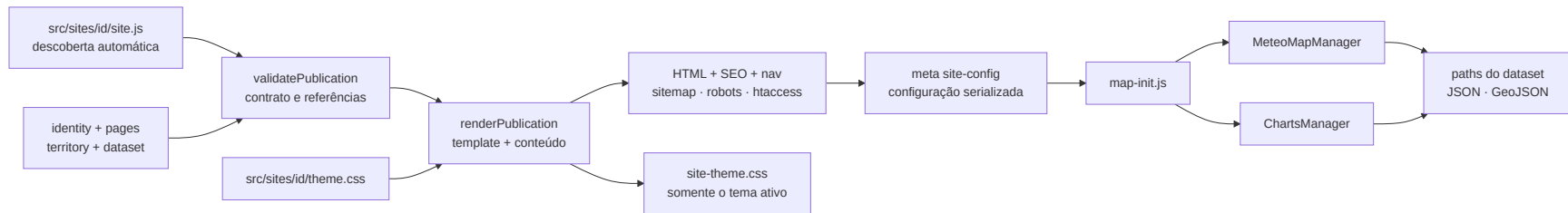
O modelo mental: quatro módulos

Módulo	Pergunta que responde	Local
Publicação	Quem publica? Qual conteúdo, SEO, navegação e marca?	<code>src/sites/<id>/</code>
Template	O que funciona igual em todos os sites?	<code>src/template/</code>
Território	Qual estado, contorno e enquadramento do mapa?	<code>src/territories/</code>
Dataset	Onde estão os dados, quais domínios e timeline?	<code>src/datasets/</code>

Um território e um dataset podem ser reutilizados por mais de uma publicação.

Evite `if (site === "ufba")` no template, no renderer, no CSS e no runtime.

Como o build compõe uma publicação



1 · Descobrir

`src/sites/*/site.js`

2 · Validar

schema, fontes, assets, GeoJSON, páginas e redirects

3 · Renderizar

HTML, tema, SEO, sitemap, robots e

`.htaccess`

Descoberta: `site.js` é a composição

```
// src/sites/ufes/site.js
"use strict";

const identity = require("../identity");
const pages = require("../pages");
const dataset = require("../../datasets/leal-wrf");
const territory = require("../../territories/es");

module.exports = { ...identity, territory, dataset, pages };
```

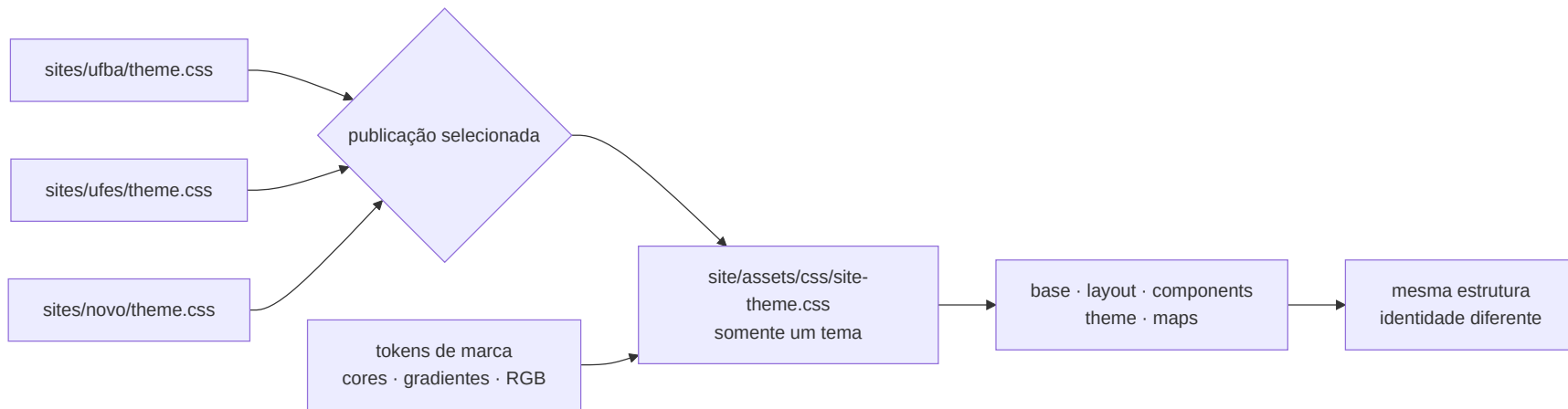
Registro automático

- uma pasta válida é o registro
- `sites:list` confirma descoberta
- `build:all` e `build:check` usam a mesma lista

Invariantes

- ID igual à pasta
- `origin` única, sem `/` final
- exatamente um `isDefault: true`
- `schemaVersion: 1`

CSS desacoplado por responsabilidade



Identidade

`src/sites/<id>/theme.css` — 19
tokens, nenhum seletor.

Estrutura comum

`base · layout · components ·
theme`

Por página

`vendorStyles + styles ; WebGIS`
declara Leaflet e `maps.css`.

Onde colocar uma mudança de estilo?

Sempre carregado

- marca → `src/sites/<id>/theme.css`
- reset/utilitário → `base.css`
- navbar/footer → `layout.css`
- cards/blocos → `components.css`
- dark mode → `theme.css`

Declarado por página

- WebGIS → `maps.css` em `styles`
- asset comum → `"assets/css/..."`
- fonte compartilhada → `templateSource("styles/...")`
- fonte exclusiva → `siteSource("styles/...")`
- biblioteca local → `vendorStyles`

Não duplique estrutura num tema e não adicione seletor por ID no CSS comum.

Comandos e saídas

```
npm run sites:list
npm run build           # publicação padrão → site/
npm run build -- --site=ufes # publicação escolhida → site/
npm run build:all       # todas → dist/<id>/
npm run build:check     # valida todas; restaura a padrão
npm run lint:themes     # contrato e isolamento dos temas
```

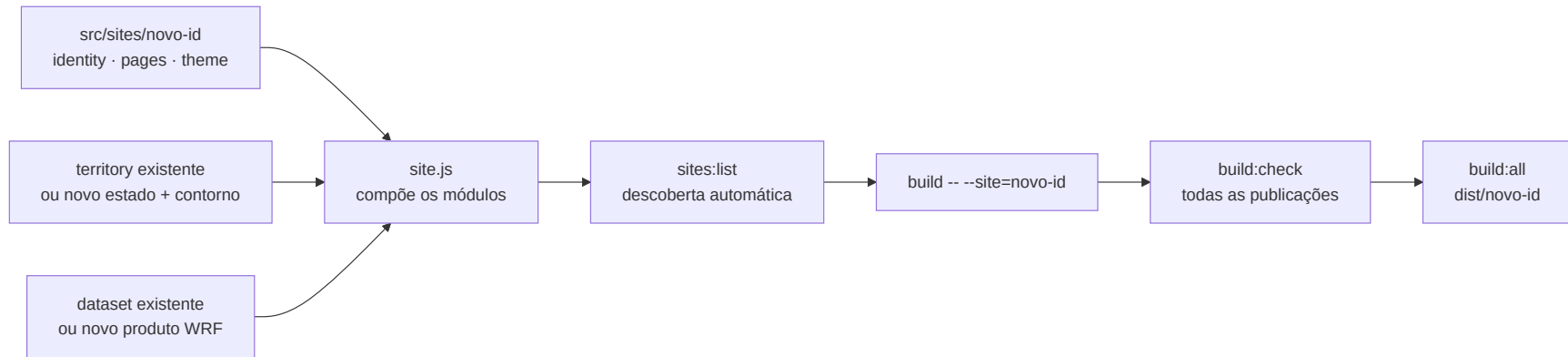
site/

- uma publicação por vez
- fluxo compatível com deploy atual
- preserva os paths operacionais do dataset
- remove HTMLs órfãos de outro site

dist/<id>/

- um frontend por publicação
- pronto para associação aos dados corretos
- omite os paths operacionais configurados
- `dist/` é git-ignored

Criar uma publicação nova: fluxo completo



Criar

```

src/sites/exemplo/
├─ site.js
├─ identity.js
├─ pages.js
├─ theme.css
├─ pages/
├─ styles/      # CSS estrutural exclusivo, opcional
└─ fragments/   # opcional
  
```

Não editar

- build.js
- package.json
- outputs derivados do manifesto
- template para trocar nomes de instituição
- CSS comum para trocar paleta

Passo 1: identidade

```
// src/sites/exemplo/identity.js
module.exports = {
  schemaVersion: 1,
  id: "exemplo",
  isDefault: false,
  origin: "https://exemplo.edu.br",
  brand: {
    name: "LAB",
    fullName: "Laboratório Exemplo",
    copyrightName: "LAB",
    ogImage: "assets/img/logo-exemplo.png",
    logos: { nav: { /* src + dimensões */ }, footer: { /* ... */ }, sidebar: { /* ... */ } },
    affiliations: [],
  },
  institution: { name: "Universidade Exemplo", acronym: "UE" },
  location: { cityName: "Cidade" },
  theme: "theme.css",
  redirects: [],
};
```

Assets apontam para `site/assets/` ; redirects só podem chegar a páginas declaradas no próprio site.

Passo 2: território e dataset

src/territories/pe.js

```
module.exports = {
  id: "pe",
  kind: "state",
  code: "PE",
  name: "Pernambuco",
  regionPhrase: "região de Pernambuco e entorno",
  terrainExample: "o Planalto da Borborema",
  boundaryAsset: "assets/data/br_pe.json",
  viewport: {
    center: [-8.3, -37.8],
    zoom: 6,
    fitBoundary: true,
    fitMaxZoom: 7,
  },
};
```

src/datasets/exemplo-wrf.js

```
module.exports = {
  id: "exemplo-wrf",
  attribution: "LAB-UE",
  paths: {
    manifest: "JSON/manifest.json",
    values: "JSON",
    grids: "GeoJSON",
  },
  timeline: {
    defaultMaxLayer: 73,
    initialIndex: 7,
    stepHours: 1,
    label: "Horário local (UTC-03)",
  },
  defaultDomain: "D01",
  domains: [/* id, label, centro, zoom, resolução */],
};
```

Montar o site

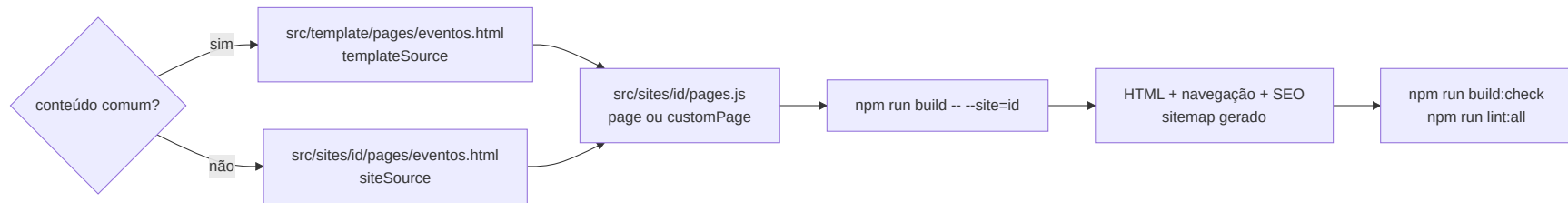
```
// src/sites/exemplo/pages.js
const { page, siteSource } = require("../../template/page-types");

module.exports = [
  page("home", {
    source: siteSource("pages/index.html"),
    seo: {
      h1: "LAB – Laboratório Exemplo",
      title: "LAB – Laboratório Exemplo · UE",
      description: "Pesquisa meteorológica da Universidade Exemplo.",
    },
  }),
  page("forecast", {
    seo: { title: "LAB – Previsões WRF · UE", description: "Previsões para Pernambuco." },
  }),
];
```

Implemente os 19 tokens em `theme.css` .

Componha tudo no pequeno `site.js` .

Adicionar uma página: primeiro decida o alcance



Compartilhada

Uma fonte em `src/template/pages/` ; cada publicação opta por incluí-la.

Exclusiva

Fonte em `src/sites/<id>/pages/` ; só aquele manifesto referencia.

Receita: página compartilhada

```
// em src/sites/<id>/pages.js
const { customPage, templateSource } = require("../../template/page-types");

customPage({
  id: "about-project",
  file: "projeto.html",
  layout: "institutional",
  source: templateSource("pages/projeto.html"),
  seo: {
    h1: "Sobre o projeto",
    title: "Sobre o projeto · LAB",
    description: "Objetivos, metodologia e resultados.",
  },
  nav: {
    label: "Projeto",
    icon: "fa-circle-info",
    order: 60,
    elementId: "nav-projeto",
  },
  styles: [templateSource("styles/projeto.css")],
});
```

Crie `src/template/pages/projeto.html` sem `<html>`, `<head>`, navbar ou footer.

Receita: página exclusiva ou variação editorial

```
const { customPage, siteSource } = require("../template/page-types");
```

```
customPage({  
  id: "projeto-local",  
  file: "projeto-local.html",  
  layout: "institutional",  
  source: siteSource("pages/projeto-local.html"),  
  styles: [siteSource("styles/projeto-local.css")],  
  seo: {  
    h1: "Projeto local",  
    title: "Projeto local · LEAL/UFES",  
    description: "Iniciativa exclusiva no Espírito Santo.",  
  },  
  nav: false,  
});
```

Anexar um trecho próprio

```
append: [siteSource("fragments/funding.html")]
```

Personalizar um tipo comum

sobrescreva `source`, `nav`, `styles` ou campos de SEO

Atualizar uma página existente: mapa de impacto

Conteúdo e catálogo

- comum → `src/template/pages/*.html`
- exclusivo → `src/sites/<id>/pages/*.html`
- anexo → `src/sites/<id>/fragments/*.html`
- SEO/H1 → `src/sites/<id>/pages.js`
- menu/ordem → `page.nav` em `pages.js`

Estrutura e identidade

- head/navbar/footer → `src/template/partials/`
- shell de página → `src/template/layouts/`
- CSS compartilhado → `site/assets/css/`
- CSS autoral → `templateSource / siteSource`
- paleta → `src/sites/<id>/theme.css`

Antes de editar algo compartilhado, procure os consumidores com `rg` e valide todas as publicações.

pages.js : uma fonte de verdade por site

Cada entrada governa:

arquivo · layout · fonte · fragmentos · styles/vendorStyles · SEO · navegação · indexação

Derivado automaticamente

- HTML completo
- item ativo da navbar
- links do rodapé
- canonical, Open Graph e Twitter
- sitemap e robots
- JSON-LD da home

Regras validadas

- IDs e outputs únicos
- uma home em `index.html`
- SEO completo
- ordem/ID/label de nav únicos
- fonte e layout existentes
- stylesheet seguro e existente

Território e dataset chegam ao runtime

```
territory + dataset  
→ renderer.runtimeConfig()  
→ <meta name="site-config" content="...">  
→ map-init.js / MeteoMapManager
```

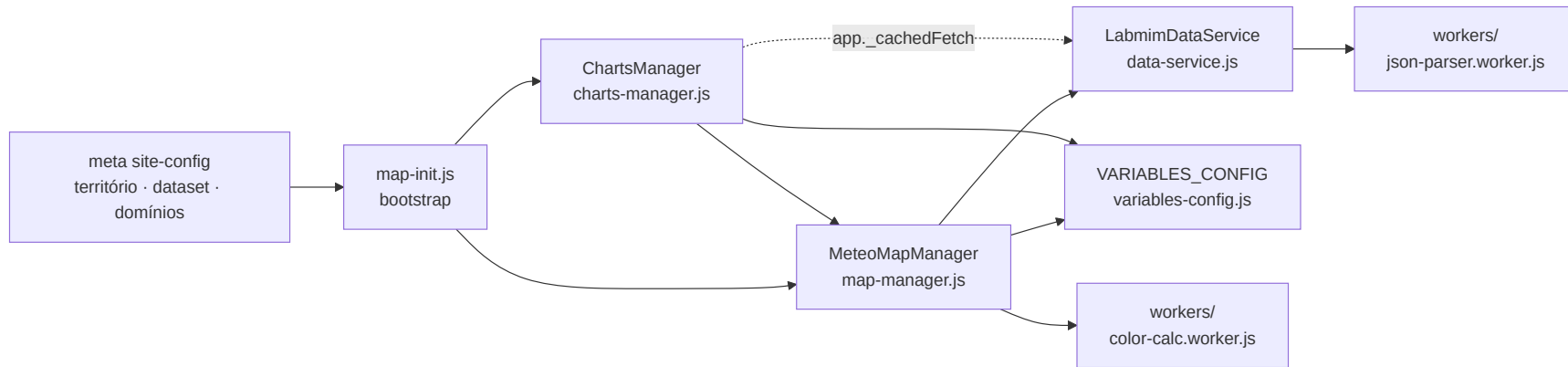
Território controla

- estado e sigla
- contorno local
- centro/zoom inicial
- `fitBounds` e zoom máximo
- textos geográficos do template

Dataset controla

- manifest, values e grids
- timeline fallback
- domínio padrão
- labels, centros e zooms
- documentação e atribuição

WebGIS compartilhado



`forecast` e `energy` reutilizam o layout WebGIS; contexto, conteúdo, estilos e SEO vêm da declaração da página.

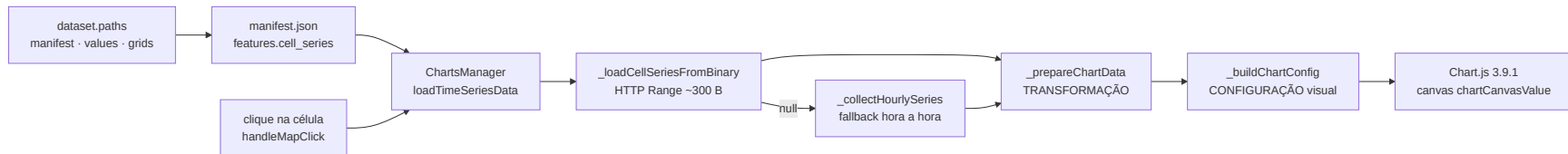
Dados e gráficos: o build não os produz

Monitoring

- 9 PNGs de nome fixo
- gerados por `labmim-site-graphs`
- cards e modais Bootstrap
- nenhuma instância Chart.js

WebGIS

- manifest + JSON/GeoJSON
- `series.bin` via HTTP Range
- `summary.json` para prévia
- Chart.js controlado por `ChartsManager`



O que `build:check` protege

- **Registro** — ID/pasta, origem, padrão
- **Identidade** — logos, URLs, redirects
- **Tema** — tokens, seletores, pares hex/RGB
- **Território** — GeoJSON, sigla, viewport
- **Dataset** — paths, timeline, domínios
- **Páginas** — fonte, layout, style, vendor, SEO/nav
- **Saída** — tokens, HTML, referências locais
- **CSS/vendor** — Bootstrap purgado e ícones

Complementar: `npm run lint:all` · `npm run format:check` · inspeção manual em light/dark e mobile.

Riscos de regressão

- editar `site/*.html` → mudança perdida
- identidade no template/CSS → marca vaza
- copiar página comum → conteúdo diverge
- `siteSource()` para capacidade comum → duplicação
- esquecer manifesto consumidor → catálogo desigual
- CSS sem `page.styles` → estilo ausente
- testar CSS comum em um site → regressão temática
- hardcode de estado/path → mapa ou dados errados



Prevenção: fronteiras declarativas + `build:check` + inspeção das publicações afetadas.

Checklists operacionais

Nova publicação

- ☐ `identity.js` , `pages.js` , `theme.css`
- ☐ páginas/fragments exclusivos
- ☐ território + contorno, se novo
- ☐ dataset, se novo
- ☐ composição em `site.js`
- ☐ `sites:list` e build individual
- ☐ `build:check` e `build:all`
- ☐ dados operacionais corretos no deploy

Página ou atualização

- ☐ decidir comum versus exclusiva
- ☐ editar fonte, não output
- ☐ atualizar SEO/nav em `pages.js`
- ☐ declarar `styles` , se houver
- ☐ verificar consumidores compartilhados
- ☐ build da publicação afetada
- ☐ `build:check` + `lint:all`
- ☐ light/dark, mobile e links

De onde vêm os dados: o repositório irmão

O `site-labmim` **não produz dados**. Ele monta HTML e associa, no deploy, os artefatos gerados por um pipeline Python separado:

- `micrometeorology` — lê a saída do modelo **WRF** e a estação, e exporta as grades, valores e figuras que o WebGIS consome.

Os dois repositórios se ligam por **contratos versionados** (manifest, grade, série por célula) e por um **contrato de id de variável**.

Para **expandir as variáveis dos mapas interativos**, o trabalho começa aqui, no pacote Python — e termina numa entrada de config no site.

As 7 CLIs `labmim-*`

Dados do WebGIS ★

`labmim-wrf-geojson` — grades, valores, `series.bin`, manifest

Figuras

`labmim-site-graphs` (9 PNGs do monitoring) · `labmim-wrf-figures` · `labmim-station-graphs`

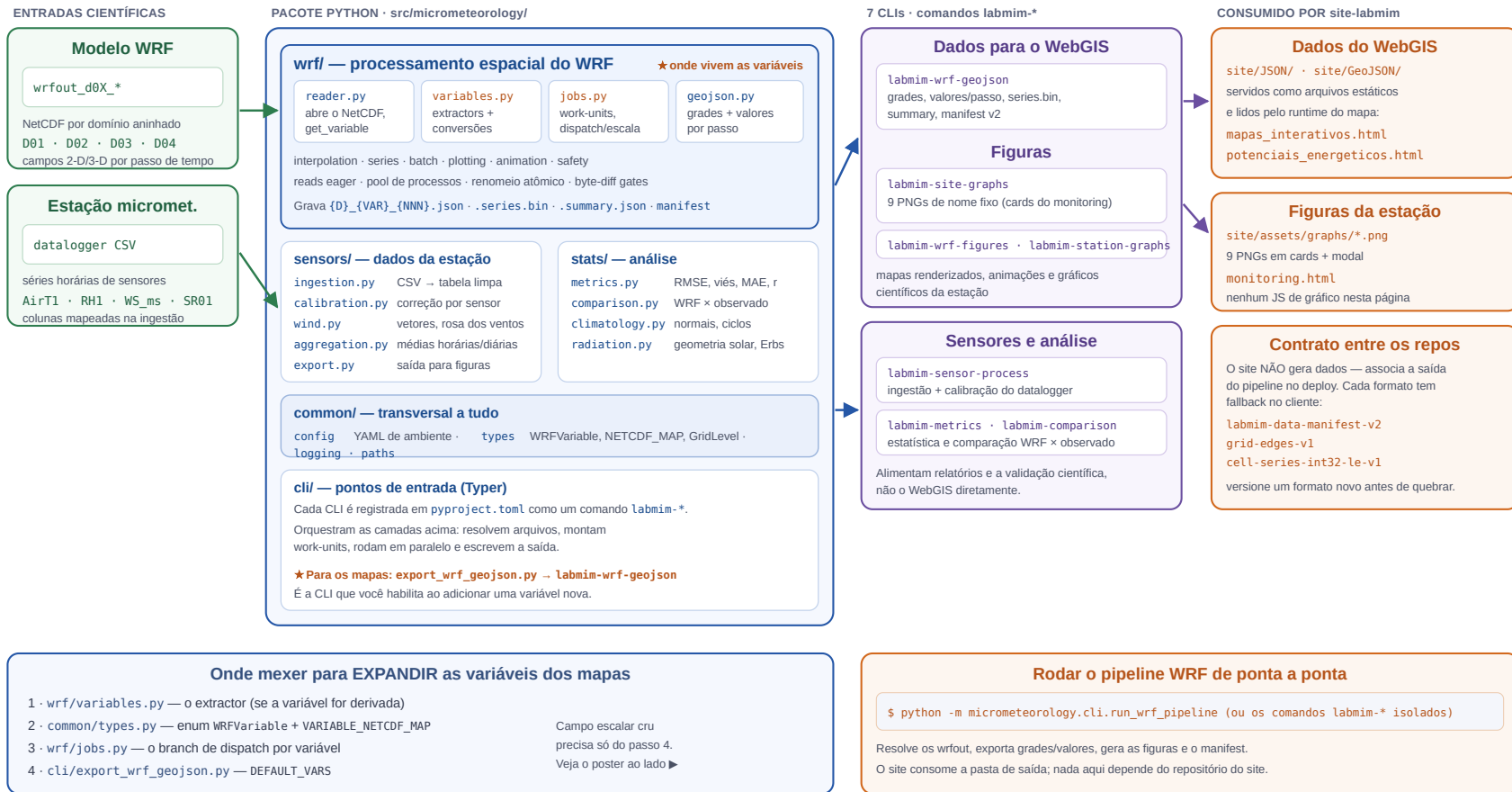
Sensores e análise

`labmim-sensor-process` · `labmim-metrics` · `labmim-comparison`

Só `labmim-wrf-geojson` alimenta os mapas interativos.

Pacote micrometeorology — do modelo ao artefato publicado

As camadas do pacote Python e as CLIs que produzem os dados e figuras do site



Adicionar uma variável nova aos mapas interativos

Do wrfout ao seletor do WebGIS — dois repositórios ligados por um contrato de id

■ micrometeorology (dados) ■ site-labmim (mapa)

REPO 1 · micrometeorology — produz os dados a partir do wrfout

1 · Que tipo de variável?

A variável já existe no arquivo wrfout_d0X como um campo 2-D pronto (escalar por passo)?

SIM → caminho rápido ▾

NÃO — precisa de fórmula, conversão ou vários campos ▾

A · Campo escalar cru — só registrar o nome

Nenhum código Python novo: o ramo genérico já cobre.

1 edição

Adicione o nome NetCDF a DEFAULT_VARS em cli/export_wrf_geojson.py
Opcional: uma cor em VARIABLE_COLORMAPS

```
# jobs._build_value_frame_source
else:
    if not ds.has_variable(
        name.upper()): return None
    extract_scalar(ds, NAME)
HFX · LH · SWDOWN · GLW
```

O contrato de id

liga os dois repositórios

```
{VAR} = VARIABLE_NETCDF_MAP
[nome] ou nome.upper()
```

O mesmo {VAR} nomeia o arquivo {D}_{VAR}_{NNN}.json E vira o sourceId no config do site.

REPO 2 · site-labmim — mostra a variável no WebGIS

B · Variável derivada — quatro edições, uma por camada

Ex.: umidade relativa (Tetens), densidade de potência eólica ($p \cdot v^3$). A física fica no docstring do extractor.

a · Extractor

wrf/variables.py
def extract_<nome>(ds)
lê campos, aplica a fórmula, retorna (vals3d, vmin, vmax)
escala via percentile_scale_bounds()

b · Tipo + id

common/types.py
membro no enum WRFVariable
entrada em VARIABLE_NETCDF_MAP
→ define o id de saída {VAR}

c · Dispatch

wrf/jobs.py
branch em _build_value_frame_source()
liga o nome ao extractor e monta o frame 2-D por passo de tempo

d · Habilitar

export_wrf_geojson.py
nome em DEFAULT_VARS (ou passar via -v <nome>)
para entrar na rodada padrão

4 · Registrar no runtime do mapa

site/assets/js/variables-config.js

a · Entrada em VARIABLES_CONFIG

id / sourceId: igual a {VAR} (o contrato)
label · unit · icon / faIcon — texto e ícone
colors[] — rampa de cor (claro → escuro)
scaleMin / scaleMax — fixam a escala do mapa
summary — texto do painel de ajuda
specificInfo(value) — cartão ao clicar a célula (sem dados → unavailableInfo(title) compartilhado)

b · Expor no seletor

Adicione a chave a VARIABLE_CONTEXTS[ctx].variables
forecast (mapas) ou energy (potenciais)

c · Aparece no mapa

seletor de variável · grade animada
série por célula · sidebar · legenda

2 · Rodar o export

```
$ labmim-wrf-geojson -v <nome> \
```

```
-o site/JSON -g site/GeoJSON -D 1,4
```

Pool de processos: um worker abre o NetCDF, deriva a variável, calcula a escala e grava todos os passos.

3 · Artefatos publicados

{D}_{VAR}_{NNN}.json	valores/ passo
{D}_{VAR}.series.bin	célula (Range)
{D}_{VAR}.summary.json	stats/ passo
manifest.json	atualizado
GeoJSON/{D}.grid.json	grade (reusada)

Armadilhas ao expandir as variáveis

▲ id que não bate

Se sourceId ≠ {VAR} do arquivo, o cliente pede um JSON que não existe → 404 e cache negativo, a variável fica muda (sem erro visual).

Confira o nome em manifest.json.

▲ ícone novo do Font Awesome

Um faIcon fora do subset atual renderiza como caixa vazia. Regenera o subset — scripts/subset-fontawesome.md

npm run build:check acusa o ícone ausente.

▲ escala e paleta

colors[] é uma rampa clara → escura; a cor por célula interpola entre scaleMin e scaleMax. Escala fixa aqui ≠ bounds por-passo do export (que usam o cap de percentil 98).

✓ campo cru = 2 edições

Escalar já no wrfout: DEFAULT_VARS + entrada no VARIABLES_CONFIG. Nada de extractor, enum ou dispatch.

A física/unidade é decisão editorial + docstring.

Receita A — campo escalar cru (2 edições)

Pipeline · `cli/export_wrf_geojson.py`

```
DEFAULT_VARS = [
    "temperature", "wind", "rain",
    # ... adicione o nome NetCDF:
    "SWDOWN", # radiação de onda curta
]
# O ramo genérico em jobs.py já cobre:
# ds.has_variable("SWDOWN") → extract_scalar
# id de saída = "SWDOWN".upper()
```

```
$ labmim-wrf-geojson -v SWDOWN \
  -o site/JSON -g site/GeoJSON -D 1,4
# → D01_SWDOWN_007.json, D01_SWDOWN.series.bin
```

Site · `site/assets/js/variables-config.js`

```
globalRadiation: {
  id: "SWDOWN", sourceId: "SWDOWN", // == {VAR}
  label: "Radiação Global", unit: "W/m²",
  faIcon: "sun",
  colors: ["#fff", "#ffd700", /* ... */ "#7a0000"],
  scaleMin: 0, scaleMax: 1200,
  summary: "Onda curta incidente na superfície.",
  specificInfo: (v) => v == null
    ? unavailableInfo("Radiação") : { /* ... */ },
},
// e expor a chave "globalRadiation" no array
// VARIABLE_CONTEXTS.forecast.variables
```

Receita B — variável derivada (uma edição por camada)

1 • Extractor · wrf/variables.py

```
def extract_relative_humidity(ds):
    q2 = ds.get_variable("Q2") # kg/kg
    t2 = ds.get_variable("T2") # K
    psfc = ds.get_variable("PSFC") # Pa
    rh = compute_relative_humidity(q2, t2, psfc)
    lo, hi = percentile_scale_bounds(rh)
    return rh, lo, hi # (vals3d, vmin, vmax)
```

2 • Tipo + id · common/types.py

```
class WRFVariable(StrEnum):
    RELATIVE_HUMIDITY = "relative_humidity"

    VARIABLE_NETCDF_MAP = {
        WRFVariable.RELATIVE_HUMIDITY: "RH2", # → {VAR}
    }
```

3 • Dispatch · wrf/jobs.py

```
if variable_name == WRFVariable.RELATIVE_HUMIDITY:
    rh, vmin, vmax = \
        variables.extract_relative_humidity(dataset)
    return _ValueFrameSource(
        frame_for_step=lambda i:
            variables.materialize_2d(rh[i : i + 1]),
        scale_min=vmin, scale_max=vmax,
    )
```

4 • Habilitar · cli/export_wrf_geojson.py

```
DEFAULT_VARS = [ ..., "relative_humidity" ]
```

Depois, no site: entrada em `VARIABLES_CONFIG` com `sourceId:` `"RH2"` (igual ao `{VAR}` do arquivo).

Contrato e armadilhas ao expandir variáveis

O contrato de id

O `{VAR}` de `{D}_{VAR}_{NNN}.json` vem de `VARIABLE_NETCDF_MAP[nome]` (ou `nome.upper()`).

O `sourceId` no site **tem** de ser esse `{VAR}` . Confira em `JSON/manifest.json` .

404 silencioso

`sourceId` errado → o cliente pede um JSON que não existe, cai em **cache negativo** e a variável fica muda, sem erro visível.

Ícone novo → regenere o subset Font Awesome (`scripts/subset-fontawesome.md`), senão vira caixa vazia.

Escala e paleta

`colors[]` é uma rampa; a cor por célula interpola entre `scaleMin` e `scaleMax` fixos no site.

Isso é **decisão editorial** — diferente dos bounds por-passo (percentil 98) que o export calcula.

Fontes: `wrf/variables.py` · `wrf/jobs.py` · `common/types.py` · `cli/export_wrf_geojson.py` · `site/assets/js/variables-config.js`

Referências rápidas

Comandos — site

```
npm run sites:list
npm run build -- --site=<id>
npm run build:all
npm run build:check
npm run lint:all
```

Comandos — pipeline de dados

```
labmim-wrf-geojson -v <var> \
  -o site/JSON -g site/GeoJSON -D 1,4
```

Guias: `src/sites/README.md` ·
`micrometeorology/README.md`

Documentação técnica

- `Architecture.md`
- `README.md`
- `docs/onboarding-architecture/architecture-evidence.md`

Código do gerador

- `scripts/site-builder/{publications,validate,renderer}.js`
- `src/template/page-types.js`

Pipeline de dados (micrometeorology)

- `wrf/variables.py` · `wrf/jobs.py`
- `common/types.py` ·